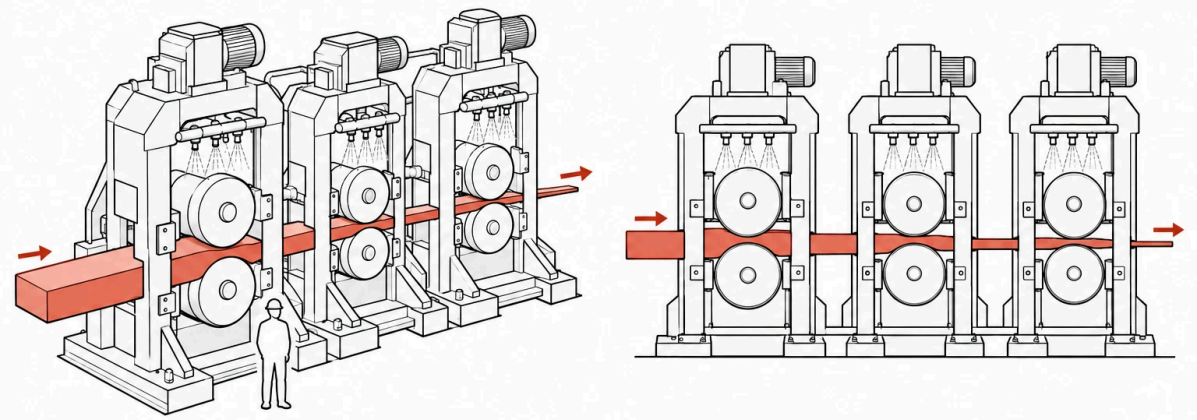
INSTITUTO FEDERAL
Minas Gerais
Campus Ouro Branco

# Middleware para supervisão térmica de cilindros na laminação TANDEM de tarugos e beam blanks

Relatório técnico

1º Trabalho Prático de Sistemas Distribuídos



Grupo TANDEM: a mesma barra passa pelas três cadeiras ao mesmo tempo.

**Gustavo Sabara Barbosa**

# Sumário


**1. Identificação** ..... **4**

**2. Objetivo e hipótese dos experimentos** ..... **5**

2.1 Objetivo ..... 5

2.2 Requisitos do enunciado ..... 5

2.3 Hipóteses dos experimentos ..... 5

**3. Arquitetura e componentes** ..... **7**

3.1 Visão geral ..... 7

3.2 Componentes ..... 7

3.3 O dado e os tópicos ..... 7

3.4 A regra de decisão ..... 8

3.5 Replicação (R7) ..... 9

3.6 Falha por queda (R8) ..... 9

3.7 Falha bizantina (R9) ..... 10

3.8 Código comum: tópicos, conexão e voto ..... 10

3.9 Código do sensor ..... 12

3.10 Código do atuador ..... 13

3.11 Código do armazenamento ..... 14

3.12 Código do controlador: a decisão ..... 16

3.13 Código do controlador: as ações do operador ..... 19

3.14 Código do controlador: recuperação e início ..... 22

3.15 Código do controlador: a réplica defeituosa de teste ..... 24

3.16 Código do cliente ..... 24

**4. Ambiente e versões** ..... **27**

**5. Procedimento de execução** ..... **28**

5.1 Preparação (uma vez) ..... 28

5.2 Execução ..... 28

5.3 Comandos dos experimentos ..... 28

**6. Resultados: tabelas, logs e medidas** ..... **29**

E1. Operação normal ..... 29

E2. Aquecimento de um cilindro ..... 29

E3. Queda de uma réplica ..... 30

E4. Réplica que reinicia ..... 30

E5. Réplica com valores errados ..... 31

E6. Duas réplicas com defeito ..... 31

**7. Discussão** ..... **33**

**8. Falhas e limitações** ..... **34**

8.1 Do modelo de falhas ..... 34

8.2 Da replicação ..... 34

8.3 Da implementação ..... 35

**9. Conclusão** ..... **36**

**10. Referências** ..... **37**

Relatório técnico do 1º Trabalho Prático de Sistemas Distribuídos.

# 1. Identificação

| Item | Conteúdo |
|---|---|
| Disciplina | Sistemas Distribuídos |
| Trabalho | 1º Trabalho Prático: implementação de um middleware |
| Autor | Gustavo Sabara Barbosa |
| Instituição | Instituto Federal de Minas Gerais, Campus Ouro Branco |
| Repositório | (preencher com o endereço do GitHub) |

**Tema.** Supervisão da temperatura dos cilindros de um grupo TANDEM de três cadeiras de laminação a quente, com aviso antes da quebra de cilindro. O material laminado são blocos de aço, como o tarugo e o beam blank, de que se obtêm perfis estruturais e outros produtos longos. No arranjo TANDEM as cadeiras ficam em sequência e a mesma barra está nas três ao mesmo tempo. A inspiração é a laminação de perfis da usina Gerdau Ouro Branco, vizinha ao campus. Nenhum dado da empresa é usado: sensores, atuadores e valores de temperatura são simulados.

**Limite do projeto.** O sistema é uma camada supervisória simulada. Não substitui CLP, intertravamento de segurança nem qualquer equipamento real.

# 2. Objetivo e hipótese dos experimentos

## 2.1 Objetivo

Construir um middleware em que sensores publicam leituras, controladores replicados decidem, atuadores executam e um cliente acompanha e age sobre o conjunto, mantendo o funcionamento correto diante da queda de um controlador e de um controlador que responde valores errados.

## 2.2 Requisitos do enunciado

| Nº | Requisito | Como é atendido | Onde |
|---|---|---|---|
| R1 | Sensores publicam no middleware | 6 sensores publicam em tópicos MQTT, um por cilindro | `sensor.py` |
| R2 | Atuadores recebem comandos pelo middleware | 5 atuadores assinam tópicos de comando | `atuador.py` |
| R3 | Controladores decidem a partir dos sensores | Regra por limite e por taxa de subida | `controlador.py` |
| R4 | Clientes visualizam e controlam o sistema | Cliente de terminal: painel, eventos, parada, rearme | `cliente.py` |
| R5 | Cliente e controlador por comunicação baseada em objetos | Objeto remoto Pyro5; o cliente não fala com o broker | `cliente.py`, `controlador.py` |
| R6 | Visualização em tempo real e ações | Sondagem de 1 s; ações de parada e rearme | `cliente.py` |
| R7 | Dados armazenados sempre replicados | 3 réplicas, cada uma com o seu banco; transferência de estado na volta | `armazenamento.py`, `controlador.py` |
| R8 | Resultado positivo mesmo com a queda de um controlador | Chamada às 3 réplicas; maioria de 2 | `cliente.py` |
| R9 | Resistir a controlador que responde valores que não conferem | Voto de 2 em 3 no cliente e no atuador | `comum.py`, `cliente.py`, `atuador.py` |
| R10 | Tema não trivial | Três cadeiras acopladas pelo mesmo material; parada conjunta | Seção 3 |
| R11 | Relatório | Este documento | `docs/relatorio.md` |
| R12 | Repositório no GitHub | Endereço na seção 1 | |

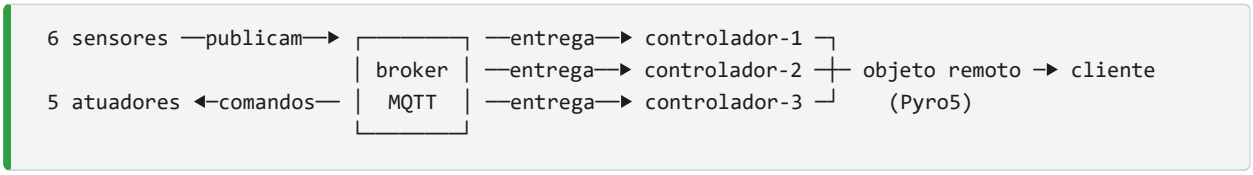
## 2.3 Hipóteses dos experimentos

| Experimento | Hipótese | Medida |
|---|---|---|
| E1. Operação normal | Uma leitura publicada chega ao cliente e fica igual nas 3 réplicas | Tempo entre a medição e a resposta ao cliente |

| Experimento | Hipótese | Medida |
|---|---|---|
| E2. Aquecimento de um cilindro | O alerta ocorre antes do limite, pela taxa de subida | Antecedência do alerta em segundos |
| E3. Queda de uma réplica | O cliente recebe resultado positivo com uma réplica parada | Chamadas com sucesso e tempo de resposta |
| E4. Réplica que reinicia | Ela volta com o mesmo estado das outras | Comparação dos armazenamentos |
| E5. Réplica com valores errados | O voto descarta a resposta divergente; o atuador não obedece | Respostas aceitas e rejeitadas |
| E6. Duas réplicas com defeito | O sistema **não** garante resultado (limite conhecido) | Comportamento observado |

# 3. Arquitetura e componentes

## 3.1 Visão geral




- Entre sensores, controladores e atuadores a comunicação é por publicar/assinar (MQTT), por um broker único.
- Entre cliente e controladores a comunicação é por invocação de métodos em objetos remotos (Pyro5). O cliente nunca fala com o broker.
- Cada controlador tem o seu armazenamento (um arquivo SQLite).
- Os controladores falam entre si só na transferência de estado, quando um deles reinicia.

## 3.2 Componentes

| Componente | Quantidade | Arquivo | Papel |
|---|---|---|---|
| Sensor de temperatura | 6 (cilindro superior e inferior de cada cadeira) | `sensor.py` | Publica uma leitura por segundo |
| Atuador: válvula de refrigeração | 3 (uma por cadeira) | `atuador.py` | Aplica a abertura decidida pela maioria |
| Atuador: alarme | 1 | `atuador.py` | Idem |
| Atuador: parada da linha | 1 | `atuador.py` | Idem; pára as três cadeiras juntas |
| Controlador | 3 réplicas iguais | `controlador.py` | Recebe, filtra, grava, decide, comanda e atende o cliente |
| Armazenamento | 1 por réplica | `armazenamento.py` | Leituras, eventos, pedidos e valores persistentes |
| Cliente | 1 ou mais | `cliente.py` | Painel e ações do operador |
| Broker | 1 | `docker-compose.yml` | Mosquitto |
| Código comum | | `comum.py` | Tópicos, endereços, log, conexão e voto |

## 3.3 O dado e os tópicos

Leitura publicada pelo sensor (JSON):

```
{"cadeira": 2, "cilindro": "superior", "temperatura": 68.8, "seq": 57,
 "instante": "2026-11-12T20:36:42.148+00:00"}
```

`seq` cresce de um em um por sensor e permite reconhecer duplicata e perda. `instante` é a hora da medição, em UTC, pelo relógio do sensor.

| Tópico | Quem publica | Quem assina |
|---|---|---|
| `laminacao/cadeira/<n>/cilindro/<c>/temperatura` | Sensor | As 3 réplicas |
| `laminacao/cadeira/<n>/refrigeracao/comando/<réplica>` | Cada réplica, no seu | Válvula da cadeira n |
| `laminacao/linha/alarme/comando/<réplica>` | Cada réplica, no seu | Alarme |
| `laminacao/linha/parada/comando/<réplica>` | Cada réplica, no seu | Parada |
| `.../aplicado` (um por atuador) | Atuador | Sensores simulados |
| `simulacao/cadeira/<n>/cilindro/<c>/falha` | Operador do experimento | Sensor simulado |

Todas as mensagens usam QoS 1 (entrega ao menos uma vez). Comandos e valores aplicados são mensagens retidas: o broker guarda a última de cada tópico e a entrega a quem assinar depois.

Alarme e parada ficam em `linha`, e não em `cadeira`, porque a mesma barra está nas três cadeiras ao mesmo tempo: não existe parar uma cadeira só.

## 3.4 A regra de decisão

Para cada leitura o controlador executa, nesta ordem: receber, filtrar pelo `seq`, gravar, decidir, comandar.

| Estado do cilindro | Condição |
|---|---|
| Crítico | Temperatura ≥ 100 °C |
| Alerta | Temperatura ≥ 80 °C, ou taxa de subida ≥ 2 °C/s com temperatura ≥ 55 °C |
| Normal | Nenhuma das anteriores |

- A taxa de subida é a diferença de temperatura entre a leitura mais nova e a mais antiga de uma janela de 5 leituras, dividida pelo tempo entre elas, tirado do campo `instante`.
- Histerese: um cilindro em alerta só volta ao normal com taxa abaixo de 0,5 °C/s.
- Do cilindro para o conjunto vale o pior caso: a cadeira tem o pior estado dos seus 2 cilindros; a linha, o pior das 3 cadeiras.

| Atuador | Valor | Quando |
|---|---|---|
| Válvula da cadeira n | 100 | Cadeira n em alerta ou crítico |
| Válvula da cadeira n | 50 | Cadeira n normal |
| Parada | 1 | Linha em crítico, ou parada pedida pelo operador |
| Alarme | 1 | Linha em alerta ou crítico, ou parada ativa |

A parada, uma vez ligada, permanece **ativa** até o operador rearmar. O rearme só é aceito com leitura dos 6 cilindros e todos normais. A parada ativa é gravada em disco antes de o comando ser publicado.

Os limites são parâmetros de simulação, não dados de usina.

## 3.5 Replicação (R7)

As três réplicas são iguais e não há chefe. Chegam ao mesmo estado porque recebem as mesmas entradas e a decisão é determinística: não usa o relógio da réplica nem sorteio.

| Entrada | Como chega às três |
|---|---|
| Leitura de sensor | O broker entrega cada mensagem a todos os assinantes |
| Ação do operador | O cliente chama o método nas três |

Uma réplica que reinicia executa a transferência de estado antes de atender:

1. anota o instante da última leitura que tem em disco;
2. conecta ao broker; as leituras novas ficam em espera;
3. pede às colegas o que perdeu: as leituras posteriores àquele instante e os pedidos já executados vêm da primeira que responder; o valor da parada ativa é pedido às duas;
4. grava o que recebeu; a leitura tem identidade `(cadeira, cilindro, instante)`, e a cópia repetida é ignorada pelo banco;
5. adota a parada ativa conforme a tabela abaixo;
6. trata as leituras em espera e passa a atender o cliente.

| Respostas das colegas sobre a parada | A réplica adota |
|---|---|
| Duas, iguais | O valor delas |
| Uma só, igual ao que ela tem em disco | Esse valor |
| Duas diferentes, ou uma só diferente do disco | Parada ativa (falha segura) |
| Nenhuma | O que tem em disco |

## 3.6 Falha por queda (R8)

- O cliente envia cada chamada às três réplicas, com tempo de espera de 3 s.
- A réplica que não responde fica **suspeita** por 10 s: as consultas a pulam. O cliente não distingue uma réplica caída de uma lenta.
- Cada ação do operador leva um identificador único, criado no cliente, o mesmo para as três réplicas e para as repetições. A réplica guarda em disco os identificadores executados e a resposta dada; um pedido repetido não é executado de novo, e a resposta guardada é devolvida.
- A garantia é **no máximo uma vez, por réplica**. Não é uma transação entre as três.

## 3.7 Falha bizantina (R9)

Uma réplica com esse tipo de falha continua respondendo, mas com conteúdo errado. Com três réplicas e no máximo uma defeituosa, as duas corretas concordam e formam maioria.

| Saída do controlador | Quem vota | Regra |
|---|---|---|
| Resposta a uma chamada | Cliente | Aceita a resposta igual em pelo menos 2 réplicas; aponta a divergente |
| Comando | Atuador | Guarda o último valor de cada réplica e aplica o sustentado por pelo menos 2 |

Cada réplica comanda num tópico próprio para o atuador saber de quem é cada voto. O atuador publica o valor que de fato aplicou.

Custo da maioria: o sistema tolera **uma** falha por vez, de qualquer tipo.

Para demonstrar, o controlador tem dois modos de teste, em que decide certo por dentro e mente nas saídas (painel, resumo, cópia para uma colega e comandos), dizendo que está tudo normal e a linha liberada:

| Comando | Comportamento | Representa |
|---|---|---|
| `python controlador.py 2 mentir` | Mente sempre | Réplica invadida |
| `python controlador.py 2 intermitente` | Mente 5 s, acerta 5 s | Erro intermitente |

## 3.8 Código comum: tópicos, conexão e voto

O arquivo `comum.py` reúne o que todos os programas compartilham. Os nomes dos tópicos são montados por funções, para que nenhum programa escreva um tópico à mão. Cada atuador tem um endereço; abaixo dele ficam os tópicos de comando, um por réplica, e o tópico do valor aplicado.

comum.py — linhas 41 a 71

```python
def topico_temperatura(cadeira, cilindro):
    return f"laminacao/cadeira/{cadeira}/cilindro/{cilindro}/temperatura"


# Um unico filtro recebe os 6 sensores ("+" vale por um nivel).
FILTRO_TEMPERATURAS = "laminacao/cadeira/+/cilindro/+/temperatura"


# Cada atuador tem um endereco. Abaixo dele ficam os topicos de comando e o do valor aplicado.
def atuador_refrigeracao(cadeira):
    return f"laminacao/cadeira/{cadeira}/refrigeracao"


# Acoes do conjunto ficam em "linha": nao existe alarme ou parada de uma cadeira so.
ATUADOR_ALARME = "laminacao/linha/alarme"
ATUADOR_PARADA = "laminacao/linha/parada"


def topico_comando(atuador, replica):
    """Cada replica comanda num topico proprio, para o atuador saber quem mandou."""
    return f"{atuador}/comando/{replica}"


def filtro_comandos(atuador):
    """Os comandos das tres replicas para um atuador."""
    return f"{atuador}/comando/+"


def topico_aplicado(atuador):
    """O valor que o atuador de fato aplicou, depois do voto."""
    return f"{atuador}/aplicado"
```

Código 1. Tópicos do processo. O `+` do filtro vale por um nível: um único filtro recebe os 6 sensores.

A conexão com o broker é criada num só lugar. As assinaturas são feitas dentro de `ao_conectar`, que a biblioteca chama a cada conexão, inclusive depois de uma reconexão. Assinar uma vez só, logo após conectar, deixaria o programa surdo depois da primeira queda do broker.

comum.py — linhas 114 a 127

```python
def conectar(nome, ao_conectar=None, ao_receber=None):
    """Conecta ao broker e devolve a conexao MQTT.

    ao_conectar: funcao chamada a cada conexao (inclusive reconexao);
                 e nela que as assinaturas sao feitas.
    ao_receber:  funcao chamada a cada mensagem recebida.
    """
    conexao_mqtt = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id=nome)
    if ao_conectar:
        conexao_mqtt.on_connect = ao_conectar
    if ao_receber:
        conexao_mqtt.on_message = ao_receber
    conexao_mqtt.connect(BROKER_HOST, BROKER_PORTA)
    return conexao_mqtt
```

Código 2. Conexão com o broker. O nome do programa é o identificador da conexão, que precisa ser único.

O voto tem dez linhas e serve aos dois lugares em que se vota. Para cada valor recebido conta quantos são iguais a ele; o primeiro que alcança 2 é devolvido. A comparação é por igualdade completa, e por isso a mesma função vota números (comandos) e dicionários inteiros (respostas ao cliente).

comum.py — linhas 76 a 86

```python
MAIORIA = 2  # de 3 replicas


def votar(valores):
    """Devolve o valor que aparece em pelo menos MAIORIA dos valores recebidos.
    Sem maioria, devolve None."""
    for candidato in valores:
        iguais = sum(1 for valor in valores if valor == candidato)
        if iguais >= MAIORIA:
            return candidato
    return None
```

Código 3. O voto. `None` significa que nenhum valor alcançou a maioria.

## 3.9 Código do sensor

O sensor é um laço: calcula a temperatura, monta a leitura, serializa em JSON e publica, uma vez por segundo. A rede roda em outra thread da biblioteca MQTT; o laço só mede e publica.

O modelo físico é mínimo e tem ordem de prioridade: linha parada, o cilindro esfria; em falha de refrigeração, aquece (menos com a válvula em 100); sem falha, volta aos poucos para a temperatura de trabalho. É simulação do mundo físico, não comunicação do processo: numa usina o sensor não ouve a válvula.

sensor.py — linhas 67 a 92

```
    while True:
        # Modelo fisico minimo, em ordem de prioridade:
        # linha parada -> esfria; falha -> sobe (menos com a valvula em 100);
        # sem falha -> volta aos poucos para a temperatura normal.
        if parada:
            temperatura += (TEMPERATURA_PARADA - temperatura) * 0.2
        elif em_falha:
            temperatura += SUBIDA_COM_REFORCO if valvula >= 100 else SUBIDA_EM_FALHA
        else:
            temperatura += (TEMPERATURA_NORMAL - temperatura) * 0.2
        temperatura += random.uniform(-0.3, 0.3)  # ruido de medicao
        temperatura = min(temperatura, TEMPERATURA_MAXIMA)

        seq += 1
        leitura = {
            "cadeira": cadeira,
            "cilindro": cilindro,
            "temperatura": round(temperatura, 1),
            "seq": seq,
            "instante": datetime.now(timezone.utc).isoformat(timespec="milliseconds"),
        }
        # Serializacao: o dicionario vira texto JSON para ser transportado.
        conexao_mqtt.publish(comum.topico_temperatura(cadeira, cilindro),
json.dumps(leitura), qos=comum.QOS)
        log.info("publicou seq=%d temperatura=%.1f", seq, leitura["temperatura"])

        time.sleep(INTERVALO)
```

Código 4. Laço do sensor: modelo físico, montagem da leitura e publicação com QoS 1.

## 3.10 Código do atuador

O atuador guarda o último valor comandado por cada réplica, identificada pelo último nível do tópico. A cada comando refaz o voto. Há três desfechos: sem maioria, mantém o valor; maioria igual ao valor atual, confirma; maioria diferente, muda o valor e publica o que aplicou.

O comando leva sempre o valor final, nunca um incremento. Receber o mesmo comando várias vezes não muda nada (idempotência).

atuador.py — linhas 50 a 67

```
50  def ao_receber(conexao_mqtt, dados, mensagem):
51      global valor_atual
52      # O ultimo nivel do topico diz qual replica mandou o comando.
53      replica = mensagem.topic.split("/")[-1]
54      votos[replica] = json.loads(mensagem.payload)["valor"]
55
56      vencedor = comum.votar(list(votos.values()))
57      if vencedor is None:
58          log.info("comando da replica %s; votos %s; sem maioria, mantido o valor %s",
59                   replica, votos, valor_atual)
60      elif vencedor == valor_atual:
61          log.info("comando da replica %s; votos %s; maioria confirma o valor %s",
62                   replica, votos, valor_atual)
63      else:
64          log.info("comando da replica %s; votos %s; maioria muda o valor %s -> %s",
65                   replica, votos, valor_atual, vencedor)
66          valor_atual = vencedor
67          publicar_aplicado(conexao_mqtt)
```

Código 5. Voto no atuador. Um comando novo substitui o anterior da mesma réplica.

## 3.11 Código do armazenamento

Cada réplica tem um arquivo SQLite com quatro tabelas. A leitura tem identidade `(cadeira, cilindro, instante)`: a mesma leitura nunca é gravada duas vezes, venha do broker ou da cópia de uma colega.

armazenamento.py — linhas 13 a 39

```python
class Armazenamento: # uso de classe pois haverão 3 controladores, 3 tabelas e 'armazenamentos' com caminhos diferentes
    def __init__(self, caminho):
        os.makedirs(os.path.dirname(caminho), exist_ok=True)
        # O banco e usado por duas threads (leituras e chamadas do cliente).
        # O controlador garante, com um mutex, que so uma o usa por vez;
        # por isso a verificacao do proprio sqlite3 pode ser desligada.
        self.banco = sqlite3.connect(caminho, check_same_thread=False)
        self.banco.execute(
            "CREATE TABLE IF NOT EXISTS leituras ("
            "cadeira INTEGER, cilindro TEXT, seq INTEGER, instante TEXT, temperatura REAL, "
            # Identidade da leitura: a mesma leitura nunca e gravada duas vezes.
            "PRIMARY KEY (cadeira, cilindro, instante))"
        )
        self.banco.execute(
            "CREATE TABLE IF NOT EXISTS eventos ("
            "instante TEXT, tipo TEXT, descricao TEXT)"
        )
        # Unica tabela em que um valor SUBSTITUI o anterior: guarda o que o
        # controlador precisa lembrar depois de cair e voltar.
        self.banco.execute(
            "CREATE TABLE IF NOT EXISTS persistente (chave TEXT PRIMARY KEY, valor TEXT)"
        )
        # Acoes do operador ja executadas: identificador do pedido e resposta dada.
        self.banco.execute(
            "CREATE TABLE IF NOT EXISTS pedidos (identificador TEXT PRIMARY KEY, resposta TEXT)"
        )
        self.banco.commit()
```

Código 6. Criação das quatro tabelas: leituras, eventos, valores persistentes e pedidos.

`INSERT OR IGNORE` descarta a segunda tentativa de gravar a mesma leitura, sem erro. Só depois do `commit` a linha está segura em disco.

armazenamento.py — linhas 49 a 56

```python
    def gravar_leitura(self, leitura):
        self.banco.execute(
            # OR IGNORE: se a leitura ja existe (veio do broker e da colega), nada muda.
            "INSERT OR IGNORE INTO leituras VALUES (?, ?, ?, ?, ?)",
            (leitura["cadeira"], leitura["cilindro"], leitura["seq"],
             leitura["instante"], leitura["temperatura"]),
        )
        self.banco.commit()  # so depois do commit a linha esta segura no disco
```

Código 7. Gravação de uma leitura.

A tabela de pedidos guarda, para cada identificador já executado, a resposta que foi dada. `ler_pedido` devolve `None` quando o identificador nunca foi visto; é por esse `None` que o controlador sabe que o pedido é novo.

armazenamento.py — linhas 92 a 103

```
92      def gravar_pedido(self, identificador, resposta):
93          """Guarda a resposta dada a um pedido (um dicionario, gravado como texto JSON)."""
94          self.banco.execute("INSERT OR IGNORE INTO pedidos VALUES (?, ?)",
95                             (identificador, json.dumps(resposta)))
96          self.banco.commit()
97
98      def ler_pedido(self, identificador):
99          """A resposta guardada para o pedido, ou None se ele nunca foi executado."""
100         linha = self.banco.execute(
101             "SELECT resposta FROM pedidos WHERE identificador = ?", (identificador,)
102         ).fetchone()
103         return json.loads(linha[0]) if linha else None
```

Código 8. Pedidos executados e suas respostas.

## 3.12 Código do controlador: a decisão

Toda a regra depende de seis números. Os estados são números em ordem de gravidade, para que "o pior caso" seja simplesmente o maior.

controlador.py — linhas 24 a 36

```
24  LIMITE_ALERTA = 80.0    # graus C
25  LIMITE_CRITICO = 100.0  # graus C
26  TAXA_ALERTA = 2.0       # graus C por segundo: entra em alerta
27  TAXA_RETORNO = 0.5      # graus C por segundo: so sai do alerta abaixo disto
28  TAXA_VALE_ACIMA = 55.0  # graus C: abaixo disto a subida e reaquecimento normal
29  JANELA = 5              # leituras usadas para calcular a taxa de subida
30
31  # Estados em numeros para que "o pior caso" seja simplesmente o maior.
32  NORMAL, ALERTA, CRITICO = 0, 1, 2
33  NOME_ESTADO = {NORMAL: "normal", ALERTA: "alerta", CRITICO: "critico"}
34
35  VALVULA_NORMAL = 50
36  VALVULA_MAXIMA = 100
```

Código 9. Parâmetros da regra de decisão. São parâmetros de simulação.

A taxa de subida divide a diferença de temperatura pelo tempo real entre a leitura mais antiga e a mais nova da janela. O tempo vem do campo `instante`, marcado pelo sensor. Por isso uma leitura perdida não distorce a taxa: a janela cobre mais tempo e o divisor aumenta junto.

controlador.py — linhas 85 a 94

```
85  def taxa_de_subida(janela):
86      """Graus C por segundo entre a leitura mais antiga e a mais nova da janela."""
87      if len(janela) < 2:
88          return 0.0
89      instante_antigo, temperatura_antiga = janela[0]
90      instante_novo, temperatura_nova = janela[-1]
91      segundos = (instante_novo - instante_antigo).total_seconds()
92      if segundos <= 0:
93          return 0.0
94      return (temperatura_nova - temperatura_antiga) / segundos
```

Código 10. Taxa de subida em graus por segundo.

A classificação testa do mais grave para o menos grave, e o primeiro teste que acerta encerra a função. Ela não usa nenhuma variável de fora, nem o relógio, nem sorteio: dadas as mesmas entradas, devolve sempre o mesmo estado. É essa propriedade que permite às três réplicas decidirem igual sem conversar.

controlador.py — linhas 97 a 110

```
97  def classificar(temperatura, taxa, estado_anterior):
98      if temperatura >= LIMITE_CRITICO:
99          return CRITICO
100     if temperatura >= LIMITE_ALERTA:
101         return ALERTA
102     # A taxa so conta perto da temperatura de trabalho: uma linha que retoma
103     # depois de parada aquece rapido de 40 para 60 graus sem haver falha.
104     if taxa >= TAXA_ALERTA and temperatura >= TAXA_VALE_ACIMA:
105         return ALERTA
106     # Histerese: quem ja estava em alerta so volta ao normal
107     # quando a temperatura praticamente parou de subir.
108     if estado_anterior != NORMAL and taxa >= TAXA_RETORNO:
109         return ALERTA
110     return NORMAL
```

Código 11. Classificação de um cilindro, com o piso de 55 °C para a taxa e a histerese.

Cada leitura passa por cinco etapas, numeradas nos comentários. O tratamento inteiro acontece com o mutex preso, porque as chamadas do cliente chegam em outra thread e mexem nas mesmas variáveis.

controlador.py — linhas 155 a 191

```
155 def ao_receber(conexao_mqtt, dados, mensagem):
156     with mutex:
157         # 1. Receber: o texto JSON volta a ser um dicionario.
158         leitura = json.loads(mensagem.payload)
159         chave = (leitura["cadeira"], leitura["cilindro"])
160         seq = leitura["seq"]
161 
162         # 2. Filtrar pelo seq.
163         ultimo = ultimo_seq.get(chave)
164         if ultimo is not None:
165             if seq == ultimo:
166                 registrar("duplicata", f"{chave} seq={seq} descartada")
167                 return
168             if seq > ultimo + 1:
169                 registrar("perda", f"{chave} faltam seq {ultimo + 1} a {seq - 1}")
170             if seq < ultimo:
171                 registrar("reinicio", f"{chave} sensor reiniciou (seq {ultimo} -> {seq})")
172                 janelas[chave].clear()
173         ultimo_seq[chave] = seq
174 
175         # 3. Gravar.
176         armazenamento.gravar_leitura(leitura)
177 
178         # 4. Decidir o estado do cilindro.
179         janela = janelas.setdefault(chave, deque(maxlen=JANELA))
180         janela.append((datetime.fromisoformat(leitura["instante"]),
    leitura["temperatura"]))
181         taxa = taxa_de_subida(janela)
182         anterior = estado_cilindro.get(chave, NORMAL)
183         novo = classificar(leitura["temperatura"], taxa, anterior)
184         if novo != anterior:
185             registrar("estado", f"{chave} {NOME_ESTADO[anterior]} -> {NOME_ESTADO[novo]} "
186                                 f"(temperatura={leitura['temperatura']}, taxa={taxa:.1f})")
187         estado_cilindro[chave] = novo
188         ultima_temperatura[chave] = leitura["temperatura"]
189 
190         # 5. Comandar.
191         atualizar_atuadores()
```

Código 12. Tratamento de uma leitura: receber, filtrar pelo `seq`, gravar, decidir e comandar.

Do cilindro para o conjunto vale o pior caso. A parada ativa é gravada em disco antes de o comando ser publicado: se o controlador cair entre os dois passos, ao voltar ele lê a parada ativa e publica o comando; na ordem inversa, voltaria religando a linha.

controlador.py — linhas 127 a 145

```
127 def atualizar_atuadores():
128     """Do cilindro para o conjunto vale sempre o pior caso (o maior).
129     Chamada depois de cada leitura e depois de cada acao do operador."""
130     global parada_ativa
131 
132     estado_linha = NORMAL
133     for cadeira in comum.CADEIRAS:
134         estado_cadeira = max(estado_cilindro.get((cadeira, c), NORMAL) for c in
    comum.CILINDROS)
135         valvula = VALVULA_MAXIMA if estado_cadeira != NORMAL else VALVULA_NORMAL
136         comandar(comum.atuador_refrigeracao(cadeira), valvula)
137         estado_linha = max(estado_linha, estado_cadeira)
138 
139     if estado_linha == CRITICO and not parada_ativa:
140         parada_ativa = True
141         armazenamento.gravar_persistente("parada_ativa", "1")
142     comandar(comum.ATUADOR_PARADA, 1 if parada_ativa else 0)
143 
144     # O alarme fica ligado enquanto a linha estiver parada aguardando o operador.
145     comandar(comum.ATUADOR_ALARME, 1 if estado_linha != NORMAL or parada_ativa else 0)
```

Código 13. Estado da cadeira, estado da linha e os comandos que decorrem deles.

Um comando só é publicado quando o valor muda, no tópico da própria réplica e como mensagem retida.

controlador.py — linhas 113 a 124

```
113 def comandar(atuador, valor):
114     """Publica o comando, no topico desta replica, somente se o valor mudou."""
115     if mentir_agora():
116         valor = VALVULA_NORMAL if "refrigeracao" in atuador else 0
117     if comandos_enviados.get(atuador) == valor:
118         return
119     comandos_enviados[atuador] = valor
120     topico = comum.topico_comando(atuador, numero)
121     # retain=True: o broker guarda o ultimo comando de cada topico e o entrega
122     # a quem assinar depois. Um atuador que liga atrasado recebe o valor vigente.
123     conexao_mqtt.publish(topico, json.dumps({"valor": valor}), qos=comum.QOS, retain=True)
124     registrar("comando", f"{topico} valor={valor}")
```

Código 14. Publicação de um comando.

## 3.13 Código do controlador: as ações do operador

Os métodos da classe exposta são os únicos que o cliente consegue chamar. Cada um pega o mutex e delega. Nas ações, a função que executa é entregue a `executar_uma_vez` sem parênteses: é passada adiante, não chamada ali.

controlador.py — linhas 291 a 297

```
291     def parar_linha(self, pedido):
292         with mutex:
293             return executar_uma_vez(pedido, executar_parada)
294
295     def rearmar(self, pedido):
296         with mutex:
297             return executar_uma_vez(pedido, executar_rearme)
```

Código 15. Métodos remotos das duas ações do operador.

`executar_uma_vez` aplica a garantia de no máximo uma vez, por réplica. Se o identificador já está guardado, a ação não é executada e a resposta antiga é devolvida. Entre executar e gravar o identificador há uma janela: uma queda nesse ponto faz a repetição ser executada de novo, e quem a torna inofensiva é a idempotência da própria ação.

controlador.py — linhas 231 a 240

```
231 def executar_uma_vez(pedido, acao):
232     """No maximo uma vez: um pedido cujo identificador ja foi executado
233     nao e executado de novo; devolve-se a resposta guardada."""
234     resposta = armazenamento.ler_pedido(pedido)
235     if resposta is not None:
236         registrar("pedido", f"{pedido} repetido; devolvida a resposta guardada")
237         return resposta
238     resposta = acao()
239     armazenamento.gravar_pedido(pedido, resposta)
240     return resposta
```

Código 16. Identificador de pedido: reconhecer a repetição.

O rearme só é aceito com leitura dos 6 cilindros e todos normais. É o único ponto do código em que a parada ativa volta a ser falsa.

controlador.py — linhas 207 a 228

```python
def executar_rearme():
    global parada_ativa
    if not parada_ativa:
        return {"aceito": True, "mensagem": "a linha ja estava liberada"}
    # So rearma conhecendo os 6 cilindros e com todos normais.
    for cadeira in comum.CADEIRAS:
        for cilindro in comum.CILINDROS:
            chave = (cadeira, cilindro)
            if chave not in ultima_temperatura:
                motivo = f"sem leitura do cilindro {cilindro} da cadeira {cadeira}"
            elif estado_cilindro[chave] != NORMAL:
                motivo = (f"cilindro {cilindro} da cadeira {cadeira} em "
                          f"{NOME_ESTADO[estado_cilindro[chave]]}")
            else:
                continue
            registrar("operador", f"rearme recusado: {motivo}")
            return {"aceito": False, "mensagem": f"recusado: {motivo}"}
    parada_ativa = False
    armazenamento.gravar_persistente("parada_ativa", "0")
    registrar("operador", "rearme aceito")
    atualizar_atuadores()
    return {"aceito": True, "mensagem": "linha liberada"}
```

Código 17. Rearme, com as três saídas: já liberada, recusado com o motivo, aceito.

O painel devolvido ao cliente é feito só de listas, dicionários, números e textos, que a serialização do Pyro5 transporta. Válvulas e alarme mostram o último valor comandado por esta réplica.

controlador.py — linhas 249 a 272

```
    def obter_painel(self):
        with mutex:
            cilindros = []
            for cadeira in comum.CADEIRAS:
                for cilindro in comum.CILINDROS:
                    chave = (cadeira, cilindro)
                    cilindros.append({
                        "cadeira": cadeira,
                        "cilindro": cilindro,
                        "temperatura": ultima_temperatura.get(chave),  # None = sem leitura
                        "estado": NOME_ESTADO[estado_cilindro.get(chave, NORMAL)],
                    })
            painel = {
                "cilindros": cilindros,
                "valvulas": [comandos_enviados.get(comum.atuador_refrigeracao(c)) for c in
comum.CADEIRAS],
                "alarme": comandos_enviados.get(comum.ATUADOR_ALARME),
                "parada": parada_ativa,
            }
            if mentir_agora():
                for cilindro in painel["cilindros"]:
                    cilindro["temperatura"] = 60.0
                    cilindro["estado"] = NOME_ESTADO[NORMAL]
                painel["parada"] = False
            return painel
```

Código 18. O painel que o operador vê.

## 3.14 Código do controlador: recuperação e início

A réplica que volta pergunta às colegas o que perdeu. Leituras e pedidos vêm da primeira que responder. A parada ativa segue a regra da seção 3.5: duas respostas iguais, adota; uma só e igual ao próprio disco, mantém; qualquer desacordo, linha parada.

controlador.py — linhas 302 a 341

```
302 def recuperar_estado(desde):
303     """Transferencia de estado: pede as colegas o que se perdeu enquanto esteve fora."""
304     global parada_ativa
305     paradas = []    # o que cada colega respondeu sobre a parada
306     copiou = False  # leituras e pedidos vem da primeira colega que responder
307     for colega in comum.REPLICAS:
308         if colega == numero:
309             continue
310         try:
311             with Proxy(comum.uri_controlador(colega)) as proxy:
312                 proxy._pyroTimeout = 3.0  # segundos de espera pela resposta
313                 copia = proxy.obter_copia(desde)
314         except CommunicationError:
315             log.info("recuperacao: %s sem resposta", comum.nome_controlador(colega))
316             continue
317         paradas.append(copia["parada"])
318         if not copiou:
319             for leitura in copia["leituras"]:
320                 armazenamento.gravar_leitura(leitura)
321             for pedido, resposta in copia["pedidos"]:
322                 armazenamento.gravar_pedido(pedido, resposta)
323             copiou = True
324             registrar("recuperacao", f"{len(copia['leituras'])} leituras recebidas de "
325                                      f"{comum.nome_controlador(colega)}")
326
327     if not paradas:
328         log.info("recuperacao: nenhuma colega respondeu; seguindo com o proprio
    armazenamento")
329         return
330     # Duas respostas iguais: adota. Uma resposta so: vale se for igual ao proprio
331     # disco (dois de acordo). Em qualquer desacordo, o lado seguro: linha parada.
332     if len(paradas) == 2 and paradas[0] == paradas[1]:
333         nova, motivo = paradas[0], "as duas colegas concordam"
334     elif len(paradas) == 1 and paradas[0] == parada_ativa:
335         nova, motivo = parada_ativa, "a unica colega que respondeu concorda com o proprio
    disco"
336     else:
337         nova, motivo = True, f"desacordo (colegas {paradas}, disco {parada_ativa}); adotado
    o lado seguro"
338     if nova != parada_ativa:
339         parada_ativa = nova
340         armazenamento.gravar_persistente("parada_ativa", "1" if parada_ativa else "0")
341     registrar("recuperacao", f"parada_ativa={parada_ativa} ({motivo})")
```

Código 19. Transferência de estado.

A ordem das quatro linhas dentro do `with mutex` importa. O instante da última leitura é lido antes de conectar; depois da conexão as leituras novas chegam, mas ficam paradas na primeira linha de `ao_receber`, à espera do mutex; só quando a recuperação termina elas são tratadas, já com a parada ativa correta. O objeto remoto é registrado por último: enquanto se recupera, a réplica não atende ninguém.

controlador.py — linhas 344 a 361

```
344  # O mutex fica preso durante todo o inicio. As leituras que chegarem do broker
345  # nesse intervalo esperam em "ao_receber" e so sao tratadas depois da
346  # recuperacao, ja com a parada ativa correta.
347  with mutex:
348      desde = armazenamento.ultimo_instante()
349      # As leituras passam a ser tratadas numa thread criada pela biblioteca MQTT.
350      conexao_mqtt = comum.conectar(nome, ao_conectar, ao_receber)
351      conexao_mqtt.loop_start()
352      recuperar_estado(desde)
353
354  # As chamadas do cliente sao atendidas aqui, na thread principal, uma por vez.
355  Pyro5.config.SERVERTYPE = "multiplex"
356  daemon = Daemon(host=comum.CONTROLADOR_HOST, port=comum.porta_controlador(numero))
357  daemon.register(ControladorRemoto(), objectId=comum.CONTROLADOR_OBJETO)
358  log.info("objeto remoto disponivel em %s", comum.uri_controlador(numero))
359
360  try:
361      daemon.requestLoop()
```

Código 20. Início da réplica: recuperação, registro do objeto remoto e laço de atendimento.

O processo tem duas threads: a da biblioteca MQTT trata as leituras, e a principal atende as chamadas do cliente, uma por vez.

## 3.15 Código do controlador: a réplica defeituosa de teste

Para demonstrar o R9 a réplica pode ser iniciada em modo de teste. Ela continua decidindo certo por dentro; só o que sai dela é falso. A função abaixo é consultada nos quatro pontos de saída: comandos, painel, resumo e cópia para uma colega.

controlador.py — linhas 53 a 59

```
53  def mentir_agora():
54      """Diz se a saida deste instante sera falsa."""
55      if MODO == "mentir":
56          return True
57      if MODO == "intermitente":
58          return int(time.time()) % 10 < 5
59      return False
```

Código 21. Modo `mentir` (sempre) e modo `intermitente` (5 em cada 10 segundos).

## 3.16 Código do cliente

Toda chamada remota do cliente passa por uma função. O `try` fica dentro do laço: a falha de uma réplica não impede a chamada às outras. Os três desfechos de uma chamada remota aparecem um em cada ramo; só a falta de resposta não tem `break`, porque é o único caso em que vale tentar de novo. A réplica que não respondeu fica suspeita, e as consultas a pulam.

cliente.py linhas 43 a 62

```
43 def chamar_em_todas(metodo, *argumentos, tentativas=1, pular_suspeitas=False):
44     """Chama o mesmo metodo nas tres replicas, uma depois da outra.
45     Devolve {numero: resposta}; a replica que nao respondeu fica de fora."""
46     respostas = {}
47     for numero, proxy in controladores.items():
48         if pular_suspeitas and time.monotonic() < suspeita_ate.get(numero, 0):
49             continue
50         for tentativa in range(1, tentativas + 1):
51             # Os tres desfechos de uma chamada remota:
52             try:
53                 respostas[numero] = getattr(proxy, metodo)(*argumentos)  # 1. devolveu o
   resultado
54                 break
55             except CommunicationError as erro:                          # 3. nao veio
   resposta
56                 log.info("controlador-%s SEM RESPOSTA em %s, tentativa %d (%s)",
57                          numero, metodo, tentativa, erro)
58                 suspeita_ate[numero] = time.monotonic() + SUSPEITA_DURA
59             except Exception as erro:                                   # 2. o metodo deu
   erro la
60                 log.info("controlador-%s devolveu um erro em %s: %s", numero, metodo, erro)
61                 break
62     return respostas
```

Código 22. Chamada do mesmo método nas três réplicas.

O voto do cliente usa a função comum e separa quem concordou de quem divergiu.

cliente.py linhas 65 a 77

```
65 def votar(metodo, respostas):
66     """Aceita a resposta dada por pelo menos duas replicas. Sem maioria, devolve None.
67     A replica que respondeu diferente da maioria e apontada no log."""
68     vencedora = comum.votar(list(respostas.values()))
69     if vencedora is None:
70         return None
71     a_favor = [numero for numero, resposta in respostas.items() if resposta == vencedora]
72     divergentes = [numero for numero, resposta in respostas.items() if resposta !=
   vencedora]
73     print(f"(maioria: {a_favor})")
74     if divergentes:
75         log.info("DIVERGENCIA em %s: controlador %s respondeu diferente da maioria %s",
76                  metodo, divergentes, a_favor)
77     return vencedora
```

Código 23. Voto nas respostas e indicação da réplica divergente.

Duas réplicas corretas podem diferir por um instante, se uma leitura chegar entre a chamada a uma e a chamada à outra. Por isso a consulta sem maioria é refeita uma vez.

cliente.py — linhas 80 a 90

```
80  def consultar(metodo, *argumentos):
81      """Consulta com voto. Duas replicas corretas podem diferir por um instante
82      (uma leitura a caminho); por isso, sem maioria, consulta mais uma vez."""
83      for vez in (1, 2):
84          respostas = chamar_em_todas(metodo, *argumentos, pular_suspeitas=True)
85          resposta = votar(metodo, respostas)
86          if resposta is not None:
87              return resposta
88          time.sleep(0.3)  # da tempo de as replicas tratarem a leitura que estava a caminho
89      log.info("%s: SEM MAIORIA (responderam: %s). Nada foi aceito.", metodo,
    sorted(respostas))
90      return None
```

Código 24. Consulta com voto e segunda tentativa.

Cada ação do operador cria um identificador, uma única vez, antes das chamadas. As três réplicas e as repetições recebem o mesmo.

cliente.py — linhas 130 a 140

```
130 def enviar_acao(metodo):
131     """Uma acao do operador: um identificador novo, o mesmo para as tres
132     replicas e para as repeticoes. O resultado e o da maioria."""
133     pedido = str(uuid.uuid4())
134     respostas = chamar_em_todas(metodo, pedido, tentativas=2)
135     resposta = votar(metodo, respostas)
136     if resposta is not None:
137         log.info("%s (pedido %s): %s", metodo, pedido[:8], resposta["mensagem"])
138     else:
139         log.info("%s (pedido %s): SEM MAIORIA (responderam: %s). "
140                  "Nao foi possivel confirmar a acao.", metodo, pedido[:8],
    sorted(respostas))
```

Código 25. Ação do operador: identificador, duas tentativas por réplica e resultado pela maioria.

# 4. Ambiente e versões

| Item | Versão |
|---|---|
| Sistema operacional | Windows 11 Home |
| Python | 3.11.9, em ambiente virtual |
| paho-mqtt | 2.2.0 |
| Pyro5 | 5.17 (serializador serpent 1.43) |
| SQLite | A que acompanha o Python |
| Docker Engine | 28.5.1 |
| Docker Compose | 2.40.2 |
| Mosquitto | 2.1.2 (imagem `eclipse-mosquitto:2`) |

Todos os processos rodam na mesma máquina. Todos os logs têm data e hora em UTC.

# 5. Procedimento de execução

## 5.1 Preparação (uma vez)

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
docker compose up -d
```

## 5.2 Execução

Um terminal para cada comando, com o ambiente virtual ativado:

```
python iniciar_controladores.py
python iniciar_campo.py
python cliente.py
```

`iniciar_controladores.py` inicia as 3 réplicas. `iniciar_campo.py` inicia os 6 sensores e os 5 atuadores. Ctrl+C encerra.

Opções do cliente: 1 ver painel; 2 acompanhar; 3 últimos eventos; 4 parar a linha; 5 rearmar; 6 comparar as réplicas.

## 5.3 Comandos dos experimentos

| Ação | Comando |
|---|---|
| Partir sem resíduo de execução anterior | `docker compose restart broker` e apagar a pasta `dados` |
| Provocar falha de refrigeração | `docker compose exec broker mosquitto_pub -q 1 -t simulacao/cadeira/2/cilindro/superior/falha -m 1` |
| Encerrar a falha | O mesmo comando com `-m 0` |
| Derrubar uma réplica | Encerrar o processo dela |
| Trazer a réplica de volta | `python controlador.py 2` |
| Réplica defeituosa | `python controlador.py 2 mentir` ou `python controlador.py 2 intermitente` |
| Ver os comandos no broker | `docker compose exec broker mosquitto_sub -v -t "laminacao/#"` |

Para os experimentos com réplica defeituosa, as três réplicas são iniciadas uma a uma, em três terminais, em vez de `iniciar_controladores.py`.

# 6. Resultados: tabelas, logs e medidas

Executados em 2026-11-12, horários em UTC. Antes de cada execução o broker foi reiniciado e os armazenamentos apagados. Os logs completos estão na pasta `experimentos`, uma subpasta por execução.

| Execução | Experimentos | Pasta |
|---|---|---|
| A | E1, E2, E3, E4 | `experimentos/A-normal-queda-retorno` |
| B | E5, réplica que mente sempre | `experimentos/B-replica-mentindo` |
| C | E5, réplica intermitente | `experimentos/C-replica-intermitente` |
| D | E6, duas réplicas mentindo | `experimentos/D-duas-replicas-mentindo` |
| E | E6, uma réplica fora e uma mentindo | `experimentos/E-uma-caida-uma-mentindo` |

## E1. Operação normal

Três réplicas corretas, 11 processos de campo, 1 cliente.

| Medida | Amostras | Mínimo | Média | Máximo |
|---|---|---|---|---|
| Tempo entre a medição no sensor e a resposta do controlador ao cliente | 20 | 9 ms | 14 ms | 20 ms |
| Duração de uma consulta ao painel nas 3 réplicas | 20 | 2 ms | 4 ms | 18 ms |

- 10 consultas ao painel pelo cliente: 10 com maioria das três réplicas.
- O que o operador vê tem atraso de até 1 s da sondagem, mais as medidas acima. O sensor mede uma vez por segundo.
- Igualdade das réplicas: verificada ao final do E4.

**Hipótese confirmada.**

## E2. Aquecimento de um cilindro

Falha de refrigeração provocada no cilindro superior da cadeira 2.

| Instante | Fato | Evidência (`controlador-1.log` e logs dos atuadores) |
|---|---|---|
| 20:36:40 | Falha provocada | |
| 20:36:42,150 | Normal → alerta a 69,0 °C | "taxa=2.3" |
| 20:36:42,171 | Válvula 2 e alarme | Válvula: votos {1: 50, 2: 100, 3: 100}, valor 50 → 100. Alarme 0 → 1 |
| 20:36:54,176 | Primeira leitura ≥ 80 °C (80,3 °C) | Armazenamento |
| 20:37:15,215 | Alerta → crítico a 100,8 °C | |

| Instante | Fato | Evidência (`controlador-1.log` e logs dos atuadores) |
|---|---|---|
| 20:37:15,242 | Parada | Votos {1: 1, 2: 1, 3: 0}, valor 0 → 1 |
| 20:37:17,211 | Cilindro de volta ao normal a 79,3 °C | "taxa=-4.9" |
| 20:37:41 | Rearme pelo operador | "linha liberada", maioria [1, 2, 3] |

| Medida | Valor |
|---|---|
| Da falha ao alerta | 2 s |
| Antecedência do alerta em relação ao limite de 80 °C | 12 s |
| Do alerta à parada | 33 s |
| Da decisão de parada ao atuador | 27 ms |

O alerta veio pela taxa de subida, 11 °C abaixo do limite de alerta. Nos dois votos acima a mudança foi aplicada com 2 votos, antes de chegar o terceiro.

**Hipótese confirmada.**

## E3. Queda de uma réplica

Réplica 2 encerrada às 20:37:52. Réplicas 1 e 3 no ar.

| Chamada | Quantidade | Com resultado positivo |
|---|---|---|
| Consulta ao painel | 15 | 15, maioria [1, 3] |
| Parada | 1 | 1: "linha parada", maioria [1, 3] |
| Rearme | 1 | 1: "linha liberada", maioria [1, 3] |

| Medida | Valor |
|---|---|
| Consulta que tenta a réplica caída | 2,05 s em média (5 amostras) |
| Consultas que tentaram a réplica caída | 2 das 15; as outras 13 a pularam por estar suspeita |
| Ação (2 tentativas na réplica caída) | 4,1 s |

Evidência (`cliente.log`): "controlador-2 SEM RESPOSTA em parar_linha, tentativa 1", "tentativa 2", e em seguida "parar_linha (pedido ffacbeed): linha parada".

**Hipótese confirmada.**

## E4. Réplica que reinicia

Réplica 2 reiniciada depois de 41 s fora, período em que perdeu uma parada e um rearme.

| Instante | Evidência (`controlador-2.log`) |
|---|---|
| 20:38:35,652 | "recuperacao: 254 leituras recebidas de controlador-1" |

| Instante | Evidência (`controlador-2.log`) |
|---|---|
| 20:38:35,668 | "recuperacao: parada_ativa=False (as duas colegas concordam)" |

Comparação pelo cliente, com os sensores parados:

| Réplica | Leituras por sensor | Total de leituras | Pedidos | Parada ativa |
|---|---|---|---|---|
| 1 | 181 em cada um dos 6 | 1086 | 3 | Não |
| 2 | 181 em cada um dos 6 | 1086 | 3 | Não |
| 3 | 181 em cada um dos 6 | 1086 | 3 | Não |

Cliente: "comparacao: as 3 replicas estao IGUAIS".

A quantidade de eventos difere (28, 26 e 28): cada réplica registra as suas próprias decisões, e a réplica 2 não registrou as do período em que esteve fora.

**Hipótese confirmada** para leituras, pedidos e parada ativa.

## E5. Réplica com valores errados

Réplica 2 defeituosa; réplicas 1 e 3 corretas. Mesma falha de refrigeração do E2.

| Medida | Mente sempre (B) | Intermitente (C) |
|---|---|---|
| Consultas ao painel | 13 | 13 |
| Aceitas, com maioria [1, 3] ou das três | 13 | 13 |
| Em que a réplica 2 foi apontada como divergente | 13 | 7 |
| Painel falso exibido ao operador | 0 | 0 |
| Válvula 2 no alerta | Votos {1: 100, 2: 50, 3: 100}; aplicado 100 | Aplicado 100 |
| Parada no crítico | Votos {1: 1, 2: 0, 3: 1}; aplicado 1 | Votos {1: 1, 2: 0, 3: 1}; aplicado 1 |
| Maior temperatura gravada | 100,1 °C | 101,0 °C |
| Comparação das réplicas com a linha parada | "DIFERENTES"; "controlador [2] respondeu diferente da maioria [1, 3]" | "IGUAIS" (consulta num intervalo em que ela acertava) |

A réplica defeituosa comandou parada em 0 e válvula em 50. A linha parou e a válvula abriu pelo voto das outras duas. A maior temperatura foi a mesma do E2, sem réplica defeituosa.

**Hipótese confirmada.**

## E6. Duas réplicas com defeito

**Caso 1: réplicas 2 e 3 mentindo, réplica 1 correta** (execução D).

| Fato | Evidência |
|---|---|
| A réplica correta detectou o crítico | `controlador-1.log` : "alerta -> critico (temperatura=101.8)" às 20:34:21 |
| A linha **não** parou | `atuador-parada.log` sem mudança de valor |
| A válvula **não** abriu | Votos {2: 50, 3: 50}: mantido 50 |
| A temperatura subiu até o teto da simulação | Maior temperatura gravada: 150,0 °C |
| O operador recebeu o painel falso | 13 consultas com maioria [2, 3] |
| A réplica correta foi apontada como a errada | "controlador [1] respondeu diferente da maioria [2, 3]" |

**Caso 2: uma réplica fora e uma mentindo** (execução E: réplica 3 não iniciada, réplica 2 em `mentir`, réplica 1 correta).

| Instante | Chamada | Resultado |
|---|---|---|
| 20:41:41 | Consulta ao painel | "SEM MAIORIA (responderam: [1, 2]). Nada foi aceito." |
| 20:41:43 | Consulta ao painel | Idem |
| 20:41:48 | Parada pedida pelo operador | Cliente: "linha parada", maioria [1, 2] |
| 20:41:44 a 20:41:51 | Atuador de parada | Votos {1: 1, 2: 0}: "sem maioria, mantido o valor 0" |

Nas consultas o sistema não aceitou a mentira: recusou-se a responder. Na ação, porém, o operador recebeu confirmação e a linha **não parou**: a réplica de teste responde às ações com verdade, o que deu maioria ao cliente, mas comanda o contrário, o que negou a maioria ao atuador.

**Hipótese confirmada:** com duas falhas simultâneas o resultado não é garantido. No caso 1 o operador é enganado e a linha não pára. No caso 2 as consultas falham de forma segura, e a ação de parada é confirmada sem ter efeito.

# 7. Discussão

**A replicação saiu quase de graça do publicar/assinar.** Nenhum código copia leituras entre réplicas em operação normal: o broker entrega a mesma mensagem às três. O que exigiu projeto foi o resto: a decisão determinística, a ação do operador enviada às três e a volta de uma réplica.

**A taxa de subida avisa antes do limite.** No E2 o alerta veio 12 s antes dos 80 °C. A taxa só vale a partir de 55 °C, porque uma linha que retoma aquece de 40 a 60 °C rápido sem haver falha; sem esse piso os testes geraram seis alertas falsos a cada retomada.

**A histerese evita oscilação.** A válvula em 100 reduz a subida para 1 °C/s, abaixo do limiar de 2 °C/s. Sem a histerese o estado voltaria ao normal, a válvula a 50, a subida a 3 °C/s, e o ciclo se repetiria.

**A ordem das operações importa em três lugares.** Gravar a parada ativa antes de publicar o comando; conectar ao broker antes de pedir a cópia a uma colega; ler o instante da última leitura antes de conectar. Em cada um, a ordem inversa produz um erro que só aparece quando o processo cai no meio.

**Duas proteções contra a repetição, e uma cobre a outra.** O identificador de pedido reconhece a repetição; a idempotência a torna inofensiva. Entre executar a ação e gravar o identificador há uma janela em que só a segunda atua.

**O voto precisa estar nas duas saídas.** Só no cliente, a tela ficaria certa e a linha exposta. Só no atuador, a linha ficaria protegida e o operador decidiria com dados falsos.

**Maioria troca disponibilidade por correção.** Com "uma resposta basta" o sistema seguiria com uma única réplica, mas aceitaria uma mentira. Com maioria ele rejeita a mentira, mas pára de responder com duas réplicas fora. Com três réplicas não se tem as duas coisas.

**A espera por uma réplica caída afeta as corretas.** Uma ligação recusada leva 2 s neste ambiente. Com as chamadas em sequência, esses 2 s separavam no tempo as respostas das duas réplicas corretas, que deixavam de coincidir. A marcação de réplica suspeita resolveu.

**Na dúvida, o lado seguro.** A réplica que volta e encontra as colegas em desacordo adota a linha parada: errar para "parada" custa produção; errar para "liberada" pode custar o equipamento.

# 8. Falhas e limitações

## 8.1 Do modelo de falhas

| Limitação | Consequência | Solução conhecida |
|---|---|---|
| Uma falha por vez | Duas réplicas defeituosas vencem o voto (E6, caso 1) | 3f+1 réplicas e protocolo de acordo |
| Broker único | Sem broker nada circula. É o desenho da figura do enunciado | Brokers em cluster |
| Broker sem autenticação | Um programa qualquer pode publicar nos tópicos de comando das três réplicas e forjar a maioria. O voto protege contra réplica **defeituosa**, não contra invasor com acesso ao broker | Usuário, senha e permissão por tópico no Mosquitto; assinatura dos comandos |
| Objeto remoto sem autenticação | Qualquer programa que conheça a URI pode parar ou rearmar | Autenticação no Pyro5 |
| Cliente, atuador e sensor são de confiança | Um sensor que mede errado é obedecido; há um sensor por cilindro | Sensores redundantes; inibição de sensor pelo operador |
| Voto de réplica caída continua valendo no atuador | Com uma réplica caída e outra defeituosa, o voto antigo pode completar a maioria errada | Validade para os votos, renovados periodicamente |
| Sem maioria, o atuador mantém o valor | Com só dois votos em desacordo, um pedido de parada não tem efeito (E6, caso 2) | Atuador de parada adotar o lado seguro quando não houver maioria |

## 8.2 Da replicação

| Limitação | Consequência | Solução conhecida |
|---|---|---|
| O atuador vota sobre o último valor de cada réplica, sem número de rodada | Quando duas réplicas corretas diferem por milissegundos, a defeituosa pode adiantar ou atrasar uma mudança por esse intervalo. Observado; o valor aplicado estava correto | Identificador de decisão em cada comando |
| A cópia de leituras e pedidos vem de uma colega só | Uma colega defeituosa pode entregar histórico falso. Só a parada ativa é conferida com as duas | Conferir a cópia com as duas colegas |
| A cópia parte do último instante gravado, de todos os sensores juntos | Uma leitura perdida de um sensor, anterior a esse instante, não é recuperada. Supõe relógios coerentes | Cursor por sensor |
| A memória de decisão não é copiada | A réplica que volta refaz janelas e estados em 5 leituras; nesse intervalo o voto das outras duas a cobre | Copiar também a janela |
| Eventos não são replicados | Cada réplica tem o registro das suas decisões; a que esteve fora tem um buraco | Copiar eventos na transferência |

| Limitação | Consequência | Solução conhecida |
|---|---|---|
| Ação que chega a duas réplicas e não à terceira, que continua no ar | A terceira fica divergente até a próxima ação ou até reiniciar. O voto a cobre | Sincronização periódica entre réplicas |
| Três réplicas que voltam juntas com discos diferentes | Nenhuma responde às outras; cada uma segue com o seu disco | Ordem de partida, ou eleição da mais atualizada |
| "No máximo uma vez" tem uma janela | Queda entre executar e gravar o identificador: a repetição é executada de novo. Coberta pela idempotência | Gravar estado e identificador na mesma transação |
| Pedido novo e atrasado é executado | Um segundo "parar" que chegue depois de um rearme pára a linha | Validade ou ordem nos pedidos |

## 8.3 Da implementação

| Limitação | Consequência |
|---|---|
| Mensagem malformada não é tratada | Um texto que não seja o JSON esperado causa erro no tratamento daquela mensagem |
| `seq` menor que o último é tratado como reinício do sensor | Uma leitura fora de ordem seria aceita como reinício |
| Eventos são exibidos sem voto | Cada réplica marca a hora pelo seu relógio; os registros nunca são idênticos |
| Painel mostra o valor comandado, não o aplicado | O cliente não lê o tópico `aplicado`, por não falar com o broker |
| Tempo real por sondagem | Até 1 s de atraso; 2 s a cada 10 s com uma réplica caída |
| Mensagens retidas sobrevivem entre execuções | Na partida o atuador recebe antes os valores antigos. O broker não tem persistência: perde-as se reiniciar |
| Tabela de pedidos só cresce | É copiada inteira a cada transferência de estado |
| Modelo físico do sensor fixado em 1 leitura por segundo | Os incrementos de temperatura são por leitura; mudar o intervalo muda a taxa simulada |
| Tudo em uma máquina | Latências e relógios não representam uma rede real |

# 9. Conclusão

O middleware atende aos requisitos funcionais do enunciado e demonstra, com medidas, os três comportamentos pedidos para os controladores: dados replicados em três armazenamentos que terminam iguais (E4), resultado positivo para o cliente com um controlador caído (E3) e rejeição das respostas e dos comandos de um controlador que mente, sempre ou de forma intermitente (E5).

A tolerância é a **uma falha por vez**. O E6 mostra o que acontece além disso, e a seção 8 lista o que o voto não cobre, em especial um invasor com acesso direto ao broker. O sistema implementa mascaramento de falha por maioria entre réplicas determinísticas; não implementa um protocolo de acordo bizantino completo.

Três réplicas votando só dão tolerância quando as corretas produzem a mesma saída. Por isso a maior parte do projeto não está no voto, que tem dez linhas, e sim no que o sustenta: decisão determinística, identidade das leituras, identificador dos pedidos e recuperação da réplica que volta.

# 10. Referências

Enunciado e material da disciplina:

- 1º Trabalho Prático de SD (enunciado) e Apêndices A, B e C do professor.

Conceitos:


- COULOURIS, G.; DOLLIMORE, J.; KINDBERG, T.; BLAIR, G. *Distributed Systems: Concepts and Design*. 5. ed. Addison-Wesley, 2011.
- TANENBAUM, A. S.; VAN STEEN, M. *Distributed Systems*. 3. ed. 2017.
- LAMPORT, L.; SHOSTAK, R.; PEASE, M. The Byzantine Generals Problem. *ACM Transactions on Programming Languages and Systems*, v. 4, n. 3, 1982.
- SCHNEIDER, F. B. Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial. *ACM Computing Surveys*, v. 22, n. 4, 1990.


Documentação das ferramentas:


- OASIS. *MQTT Version 3.1.1*. https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html
- Eclipse Mosquitto. https://mosquitto.org/documentation/
- Eclipse Paho, cliente Python. https://eclipse.dev/paho/
- Pyro5. https://pyro5.readthedocs.io/
- SQLite. https://www.sqlite.org/docs.html
- Python 3.11. https://docs.python.org/3.11/
- Docker Compose. https://docs.docker.com/compose/